UCAM

Práctica final

Cálculo paralelo de gradientes de funciones multivariables con apoyo de inteligencia artificial generativa en el desarrollo

Programación Paralela

Baldomero Imbernón Tudela

Grado en Ingeniería Informática



Índice de contenidos

1. Objetivos 2
2. Enunciado 2

2.1 Tareas a desarrollar *5*
2.1.1 Paralelización en procesadores con memoria compartida en CPU (40%) 5
2.1.2 Paralelización en dispositivos preparados para computación de alto rendimiento (GPUs) usando CUDA. (40%) 6
2.1.3 Documentación. (20%) 7
2.1.4 Fechas de entrega 8
2.1.5 CONVOCATORIA DE RECUPERACIÓN JUNIO-JULIO 2026 8

3. Referencias bibliográficas 9

1. Objetivos

El objetivo de esta práctica es que el alumnado implemente optimice y compare diferentes estrategias de programación paralela (basadas en OpenMP y CUDA) para la resolución de sistemas de ecuaciones lineales multivariables, abordando además el uso de herramientas de inteligencia artificial generativa para apoyar el desarrollo del código. El enfoque se centrará tanto en el rendimiento como en la comprensión de cómo interactuar eficazmente con modelos de IA mediante el diseño cuidadoso de *prompts*.

2. Enunciado

El cálculo de gradientes de funciones multivariables es una operación central en numerosos algoritmos de optimización, aprendizaje automático y simulación científica. Desde el entrenamiento de redes neuronales hasta la resolución de problemas de mínimos cuadrados, la computación eficiente de gradientes en espacios de alta dimensión es esencial en aplicaciones modernas de inteligencia artificial, visión por computador y física computacional.

En este contexto, la programación paralela ofrece una vía para mejorar significativamente el rendimiento computacional de estos cálculos, especialmente cuando el número de variables o de evaluaciones de la función es elevado. Tecnologías como **OpenMP** (para paralelismo en CPU) y **CUDA** (para paralelismo en GPU) permiten aprovechar los recursos de hardware actuales para acelerar estos procesos.

Simultáneamente, herramientas de IA generativa con modelos basados en GPT, Claude, LLaMA, Gemini, Falcon, Cohere, entre otros, están transformando el desarrollo de software. Estas herramientas pueden asistir en tareas como la redacción de algoritmos, explicación de errores, optimización de código, documentación o generación de casos de prueba. Sin embargo, su efectividad depende en gran medida de la forma en que se les formula la consulta. Esta habilidad, conocida como **ingeniería de prompts**, requiere práctica, reflexión crítica y comprensión técnica para guiar correctamente a los modelos de lenguaje. Por ello, esta práctica se centra en dos objetivos fundamentales:

1. **Desarrollar una implementación eficiente y paralela** para el cálculo del gradiente de funciones arbitrarias.
2. **Explorar el uso de herramientas de IA generativa** para asistir en el diseño, implementación y optimización del código **DINÁMICO**, enfatizando la importancia del diseño de *prompts* adecuados.



INSTRUCCIONES METODOLÓGICAS OBLIGATORIAS

1. **Seguimiento estricto del método:** Los alumnos **deben seguir los pasos matemáticos y algorítmicos detallados en el siguiente ejemplo** para estructurar la resolución de la práctica.
2. **Generalización (Múltiples monomios y variables):** El ejemplo que se adjunta a continuación es **únicamente un caso ilustrativo simplificado**. El código por desarrollar debe ser totalmente **dinámico, modular y extensible** para soportar funciones compuestas por **múltiples monomios y/o múltiples variables**.
3. **El CODIGO debe ser alojado en el servidor de prácticas, y las pruebas deberán realizarse sobre ese entorno hardware, de cara a la entrega final de la práctica.**
4. **Puntualización sobre Múltiples Variables / Sistemas:**
    - Los apartados **1.1.1** y **1.1.2** se centrarán en la evaluación e implementación paralela del gradiente de **una función escalar multivariable arbitraria** (generalizando el ejemplo a N variables y M monomios).
    - La extensión del cálculo del gradiente/Jacobiano a **sistemas de ecuaciones multivariable** (múltiples ecuaciones simultáneas) **se contempla de manera explícita y exclusiva en el apartado 1.1.3**.

Ejemplo práctico de referencia (Paso a Paso)

A continuación, se ilustra el procedimiento analítico que debe servir de guía para la resolución algorítmica:

Sea la función:

f(x, y, z) = 3xy + 4yz + 2x + 5

PASO 1. Derivadas parciales

Respecto a x:
∂f/∂x = ∂/∂x(3xy + 4yz + 2x + 5) = 3y + 0 + 2 + 0 = **3y + 2**

Respecto a y:
∂f/∂y = ∂/∂y(3xy + 4yz + 2x + 5) = 3x + 4z + 0 + 0 = **3x + 4z**

Respecto a z:
∂f/∂z = ∂/∂z(3xy + 4yz + 2x + 5) = 0 + 4y + 0 + 0 = **4y**

PASO 2. Formulación del Gradiente de f(x, y, z)

∇f(x, y, z) = (∂f/∂x, ∂f/∂y, ∂f/∂z) = **(3y + 2, 3x + 4z, 4y)**

PASO 3. Evaluación numérica en un punto especificado (x, y, z) = (1, 2, 3)

∂f/∂x = 3·2 + 2 = 8

∂f/∂y = 3·1 + 4·3 = 3 + 12 = 15

∂f/∂z = 4·2 = 8

∇f(1, 2, 3) = (8, 15, 8)

(Recuerda: Tu programa debe automatizar este procedimiento paso a paso para cualquier función compuesta por múltiples monomios y variables).

2.1 Tareas a desarrollar

1. **Diseñar una función C** que represente dinámicamente una función arbitraria (soporte para múltiples monomios y variables) y calcule numéricamente su gradiente.

2. **Paralelizar el cálculo del gradiente** en dos versiones:

   - **OpenMP**: para aprovechar múltiples núcleos de CPU.

   - **CUDA**: para ejecutar el cálculo de gradientes de manera masiva en GPU.

3. **Comparar el rendimiento** de ambas versiones respecto a la secuencial, variando el número de hilos y tamaños de problema y tamaños por bloque en la GPU.
4. **Aplicar la IA generativa como herramienta de apoyo al desarrollo**, documentando interacciones relevantes y **cómo los *prompts* fueron afinados para obtener mejores resultados en todas las fases, desde la generación de código secuencial, hasta optimizaciones paralelas.**
5. **EXTENDER la solución del cálculo de gradientes a sistemas de ecuaciones multivariable. (exclusivo del punto 1.1.3).**
6. **INCLUIR la solución del cálculo de gradientes en un sistema de contenedores.**

2.1.1 Paralelización en procesadores con memoria compartida en CPU (30%)

El alumno deberá paralelizar el cálculo del gradiente utilizando OpenMP (Chandra, 2001). El punto de partida será la implementación secuencial (que generaliza el paso a paso del ejemplo para múltiples monomios/variables), la cual servirá de guía para comprobar la corrección de los resultados. Requisitos:

- **Calcular el tiempo de ejecución (se aconseja utilizar `omp_get_wtime()`).**

- **Análisis de escalabilidad.** Analizar la ejecución para distinto número de hilos y realizar un análisis crítico de los resultados obtenidos.

- **Estudio de paralelismo anidado.** Estudiar la distribución de hilos entre los diferentes bucles anidados (evaluación de variables vs. evaluación de monomios/puntos).

2.1.2 Paralelización en dispositivos preparados para computación de alto rendimiento (GPUs) usando CUDA. (20%)

El alumno deberá realizar una implementación en CUDA del algoritmo de cálculo de gradientes propuesto.

Requisitos mínimos exigidos:

1. **Implementación CUDA correcta del algoritmo en cuanto a resultados con respecto a la versión secuencial**. (*Si este punto no se completa no se irá avanzando en la corrección de los distintos apartados*)
2. **Comparación de rendimiento con el código secuencial para los ficheros de entrada elaborados por el alumno y/o cualquier otro método que estime el alumno para aportar los datos.**
3. **Desglose de tiempos de ejecución del código CUDA**. División entre tiempos de transferencia de datos entre CPU y GPU, tiempos de cómputo y tiempo total de la ejecución.
4. Conclusión justificada de los tiempos obtenidos

Parte opcional (Optimización):

En este apartado optimizaremos el/los *kernels* obtenido en el punto anterior. Para realizar esta optimización es fundamental realizar un análisis profundo de la ejecución en la GPU.

A continuación, se dan algunas pistas de posibles optimizaciones a este problema.

- **Ocupación de SMs.** Examina la información obtenida de ptxas=1 que describe el número de registros utilizados por hilos, la cantidad de shared y constant memory utilizada. Puedes utilizar la hoja de cálculo de NVIDIA (CUDA_Occupancy_calculator.xls) para ver cómo mejorar el número de bloques activos en cada multiprocesador.
- Aprovechar **la localidad espacial** a través de la técnica de tiling con la memoria compartida de los multiprocesadores.
- Otros tipos de estrategias para reducir la presión en *device memory* como, uso de memoria de texturas, incremento del uso de los registros (*register tiling*), *memory coalescing, memory camping*, etc. para optimizar al máximo tú código.

- También puedes replantear el problema a nivel algorítmico para obtener un patrón de cómputo más adecuado a la arquitectura de la GPU.

2.1.3 Extender la solución a SISTEMAS DE ECUACIONES MULTIVARIABLE. (20%)

En este apartado específico, se pide extender la solución **paralelizada desarrollada anteriormente** para dar soporte al cálculo simultáneo del gradiente (matriz Jacobiana) en **sistemas de ecuaciones multivariable** (múltiples funciones con múltiples variables y monomios)

Ejemplo:

$3x^2 + yz + z + 5 = 2$
$2x + yz^3 + 2x + 2 = 0$
$3xz + 8yz + z^2 + 1 = 4$

El alumno deberá adaptar las estructuras de datos, las regiones paralelas de OpenMP y los *kernels* de CUDA para procesar el conjunto completo de ecuaciones de forma eficiente.

2.1.4 Uso de promtps y modelos de IA. (15%)

La documentación aquí propuesta puede incluirse dentro de uno de los puntos de la documentación general, pero deberá consistir en un análisis de los prompts y de las respuestas generadas de las IAs utilizadas para desarrollar la práctica, de manera que se muestre de manera incremental, como se ha ido refinando el código proporcionado por la IA hasta la solución final. Para cada una de las IAs utilizadas:

1. Ámbito de uso de la IA. *(es decir, en que parte la habéis usado, desde el principio o más adelante con un código previamente proporcionado por otra IA).*
2. Lista de los prompts. *(Se va a puntuar la calidad y el contexto de los mismos).*
3. Diferentes soluciones proporcionadas. *(Se puede dar una descripción de la misma y aportar las partes de código más importantes que han sido ofrecidas).*
4. Valoración final.

2.1.5 Documentación GENERAL. (15%)

La documentación generada deberá seguir el siguiente esquema. Para sacar la información de cada apartado, se puede utilizar el contenido de las citas bibliográficas presentes en este documento, con su correspondiente cita.

1. Abstract
2. Introducción

3. El problema
4. Paralelización del algoritmo
    a. Descripción del código secuencial
    b. Paralelización en procesadores multinucleo
    c. Paralelización en tarjetas gráficas
    d. Extensiones
5. Resultados experimentales
    a. Entorno de simulación
    b. Resultados
6. Conclusiones
7. Bibliografía.

2.1.6 Fechas de entrega

Fecha de entrega final: **TAREA DE ENTREGA**

1. Entrega opcional valoración del código (OpenMP). ***TAREA DE ENTREGA***.

La valoración de cada apartado está descrita en el enunciado y se podrá solicitar una entrevista por parte del profesor para comprobar la realización de la misma, siendo el resultado de la entrevista, la valoración final de la práctica.

2.1.7 CONVOCATORIA DE RECUPERACIÓN JUNIO-JULIO 2026

Para la convocatoria de recuperación se requerirá de **MANERA OBLIGATORIA** la implementación **PARCIAL O TOTAL** de la práctica en **CUDA**. **Aunque por supuesto que la práctica debe funcionar de igual forma que en código secuencial**. En función de la parte paralelizada, se establecerá nota final de la parte práctica.

3. Referencias bibliográficas

Appel, K., Haken, W., & Koch, J. (1977). Every planar map is four colorable. Part II: Reducibility. Illinois Journal of Mathematics, 21(3), 491-567.

Babai, L. (2016). Graph isomorphism in quasipolynomial time. In Proceedings of the forty-eighth annual ACM symposium on Theory of Computing (pp. 684-697).

Brownlee, J. (2023). *Prompt Engineering for Coders*. Machine Learning Mastery.

Buitrago, D. (2016). A sociometric approach applied to the description of a social network in a personalized education school in Bogotá. International journal of sociology of education.

Chandra, R. (2001). Parallel programming in OpenMP. Morgan kaufmann.

Kolomeets, M., Chechulin, A., & Kotenko, I. V. (2019). Social networks analysis by graph algorithms on the example of the VKontakte social network. J. Wirel. Mob. Networks Ubiquitous Comput. Dependable Appl., 10(2), 55-75.

Floyd, R. W. (1964). Algorithm 245: treesort. Communications of the ACM, 7(12), 701.

Gao, Y., Hasegawa, H., Yamaguchi, Y., & Shimada, H. (2022). Malware Detection by Control-Flow Graph Level Representation Learning With Graph Isomorphism Network. IEEE Access, 10, 111830-111841.

He, J., Chen, J., Huang, G., Cao, J., Zhang, Z., Zheng, H., ... & Van Zundert, A. (2021). A polynomial-time algorithm for simple undirected graph isomorphism. Concurrency and Computation: Practice and Experience, 33(7), 1-1.

Heckmann, T., Schwanghart, W., & Phillips, J. D. (2015). Graph theory—Recent developments of its application in geomorphology. Geomorphology, 243, 130-146.

Mattson, T. G., Sanders, B. A., & Massingill, B. L. (2004). *Patterns for Parallel Programming*. Addison-Wesley.

OpenAI (2023). *Best practices for prompt engineering with AI models*. https://platform.openai.com/docs/guides/gpt-best-practices

Pitts, F. R. (1965). A graph theoretic approach to historical geography. The professional geographer, 17(5), 15-20.

Wu, L., Cui, P., Pei, J., Zhao, L., & Guo, X. (2022). Graph neural networks: foundation, frontiers and applications. In Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (pp. 4840-4841).